INSIDE AI · 金智维

SDAFI

业务 Agent 运行时架构白皮书

从 PPT 概念,到可运行的代码

架构总结 · 核心能力 · Audit-by-Design 专题

2026

2024–2026 Agent 浪潮,但严肃业务里几乎没跑起来

四个现实问题,决定了通用 Agent 框架跨不过能源 / 金融 / 政务的门

01 PPT 概念 vs 可运行代码

Demo 惊艳、发布会酷炫,进入生产环境就崩——大多数 Agent 项目停留在「能演示一次」的阶段。

02 演示场景 vs 监管场景

能源 / 金融 / 政务要的不只是「能跑」,还要「能审」——每次决策要能回溯到输入、过程、影响。

03 单次对话 vs 长期演化

真实业务不是一次问答,是每天 7×24 跑的系统——状态要能持久、能恢复、能随着数据自己变好。

04 涌现能力 vs 业务正确性

严肃业务不能靠「Agent 之间自己聊出来」——每次交互都必须是结构化、可预期、可审计的。

目录

Contents

01 **定位与问题**
开场钩子 · SDAFI 是什么 · 四大断点 P02–P05

02 **核心设计哲学**
五阶段 · 三触发 · 数据池介质 · 架构全景 P06–P09

03 **运行时与架构亮点**
运行时结构 · 九大架构亮点 P10–P13

04 **Audit-by-Design 专题**
证据链 · 审计对比 · 强监管场景 P14–P17

05 **验证与对比**
同类框架对比 · solar_forecast 闭环 P18–P19

06 **落地与展望**
成熟度数据 · 定位金句 · 合作联系 P20–P22

一页纸定位

What is SDAFI

SDAFI 不是通用 Agent 框架,是为严肃业务场景量身定制的**业务 Agent 运行时。**

每个 Agent 是「会思考、有节律、可被唤醒」的业务领域对象,而不是 LLM 的壳。

会思考

S→D→A→F→I 五阶段生命周期,每轮循环自主决策

有节律

心跳 / 消息 / 订阅 三位一体,原生支持多时间尺度

可被唤醒

数据池 + LISTEN/NOTIFY,Agent 零直接调用全解耦

严肃业务的四大断点

通用 Agent 框架在能源 / 金融 / 工业场景落地时的系统性缺失

01 时间尺度单一

只能响应用户请求,不支持心跳兜底、订阅推送、异步信号这类业务固有的多时间尺度协同。

02 状态瞬态易失

Chain / Graph 执行完即销毁,进程崩溃就丢,无法支撑需要长期演化的业务 Agent。

03 协作难以审计

Agent 间通过自然语言协商,事后无法结构化回溯「当时基于什么数据做的决策」。

04 无自我演化

Agent 是无状态执行器,每次调用从零开始,缺少「看效果、调参数」的在线校准能力。

五阶段生命周期 · S→D→A→F→I

每个 Agent 每个 cycle 跑一遍完整的感知—决策—行动—反馈—改进循环

S	D	A	F	I
Sense	**Decide**	**Act**	**Feedback**	**Improve**
读数据池 + 消费信号	规则 / 模型 / LLM 决策	写自己的数据池 + 发信号	验证上一轮决策效果	调整内部参数
纪 律	纪 律	纪 律	纪 律	纪 律
只读 · 纯 IO	*不碰 DB*	*唯一允许写*	*量化 deviation*	*让下一轮更准*

下一轮循环 ←



关键设计:只有 Act 阶段能写,其他阶段严格只读。这条纪律让 多个Agents 协同时, 路径清晰可追踪、可回滚、可审计。

三种触发机制 · 时间作为一等公民

心跳 / 消息 / 订阅 三位一体,原生支持多时间尺度协同



消息

Message

类比

神经反射

时间尺度

毫秒 — 秒级

主要用途

点对点 Agent 协作

典型场景

设备告警 → 应急响应 Agent



订阅

Subscription

类比

血液循环

时间尺度

秒 — 分钟级

主要用途

一对多数据推送 (pub/sub)

典型场景

价格更新 → 所有策略 Agent



心跳

Heartbeat

类比

脉搏

时间尺度

30 秒 — 5 分钟

主要用途

慢循环兜底

典型场景

日结算 · 周期对账 · 慢刷新

数据池作为唯一介质

Agent 之间零直接调用,所有跨 Agent 信息流都通过共享表



解耦彻底

Agent 之间不知道彼此的存在,替换 / 新增 Agent 不影响其他

天然可追溯

所有状态变更 + 信号流转在 PG 里留痕,支持任意时刻回放

多实例无状态

任一实例崩溃,状态都在 PG,重启即恢复,支持横向扩展

容灾与审计双重红利

为强监管行业(能源 / 金融 / 政务)提供合规级保障

架构全景 · 三层模型

业务层 · 运行时层 · 存储层 —— 一张图看懂 SDAFI 的整体结构

一、业务层 · 14 个业务 Agent

设备感知 Agent	YAML 驱动 · 配置即 Agent · 新增无需改框架
策略决策 Agent	规则 + 模型 + LLM · 按场景自由组合
二、运行时层 · SDAFI 五阶段	S→D→A→F→I 闭环 · 生命周期调度 · 跨站协同
├─ **三种触发机制**	心跳 · 消息 · 订阅 · 多时间尺度协同
├─ **启动时契约校验**	Fail Fast · 配置错误上线前暴露
三、存储层 · PostgreSQL	业务表 · 信号日志 · 15+ TimescaleDB hypertables
└─ **数据池 + LISTEN/NOTIFY**	Agent 间零直接调用 · 所有交互必经数据池 · 跨实例唤醒

三层六特性

- 业务层 · 声明式配置 · YAML 即 Agent
- 业务层 · 异构决策 · 规则 / 模型 / LLM 自由混合
- 运行时层 · 五阶段生命周期 · 自主决策闭环
- 运行时层 · 时间一等公民 · 心跳 / 消息 / 订阅
- 存储层 · 零直接调用 · 数据池为唯一介质
- 存储层 · Audit-by-Design · 证据链架构副产物

运行时架构总览

main.py lifespan 启动的组件结构

main.py · lifespan	
YAMLAgentLoader	14 个配置 → AgentConfig
HandlerPipelineRunner	inspect 签名自适应 · 每 phase 独立 tx
AgentOrchestrator	per-station / batch 策略 · 跨站循环
├─ TriggerManager	asyncio.Event + PG LISTEN/NOTIFY 双轨
├─ _signal_cleanup	TTL 清理 + Prometheus gauge 采样
└─ _retention_loop	daily cycle_log / health_monitor 保留
SimulatorManager	6 个数据源背景任务

设计原则

- 声明式配置 · YAML 驱动,新 Agent 无需改框架
- 契约校验前置 · uvicorn 启动即检查签名合法性
- 短事务 opt-in · LLM 调用不堵数据库连接池
- 参数渐进式扩展 · inspect.signature 零 breaking
- 信号自动发现 · checkpoint 机制免手写 emit 列表
- 无状态多实例 · PG LISTEN/NOTIFY 跨进程唤醒

九大架构亮点

每一条都对应一个具体的工程取舍,从 15 次架构整修中沉淀

01 每 phase 独立短事务

session_factory opt-in,LLM 不堵 DB 池

02 inspect 签名自适应

handler 参数渐进扩展,零 breaking change

03 信号目标自动发现

checkpoint 机制消除「忘写就静默睡」bug

04 execution_strategy

单架构覆盖 L0–L3,YAML 一行切换

05 LISTEN/NOTIFY 无状态

多实例唤醒免 Redis / MQ,PG 自带能力

06 启动时契约校验

配置错误在 uvicorn 启动就抛,不延后

07 五阶段完整闭环

solar_forecast 真实数据收敛验证

08 可观测性原生

Prometheus 4 大指标开箱即用

09 Audit-by-Design

审计证据链原生落地(专题展开)

亮点深化 · 单架构覆盖 L0–L3 · 无状态多实例

一套运行时,既能跑细粒度设备感知,又能跑聚合分析

execution_strategy · 一行 YAML 切换

L0 / L1 · per-station

device-sensing · 每站跑一次

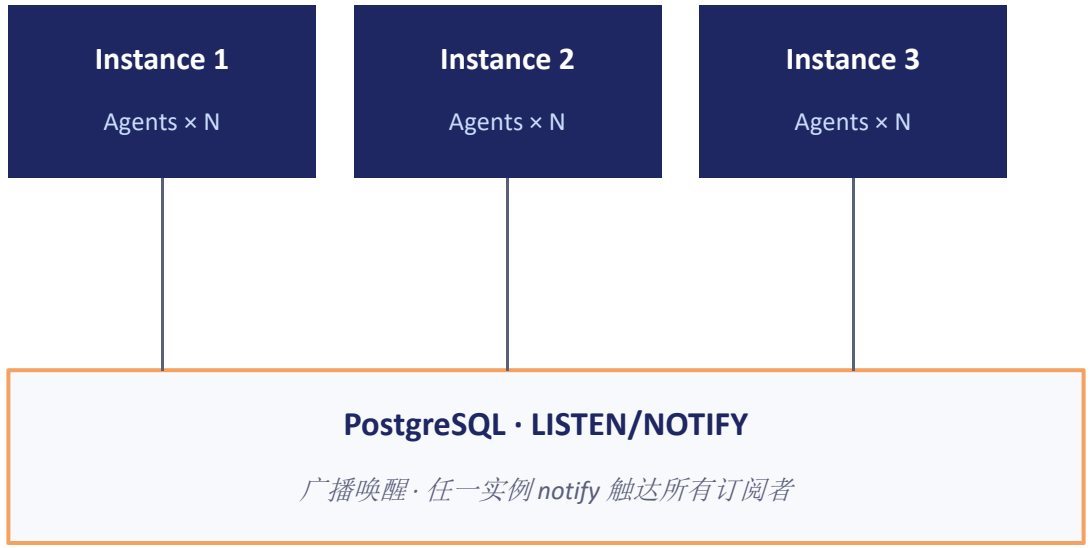
```
execution_strategy: per_station
for station in stations:
    run_pipeline(station)
```

L3 · batch

area-analyst · 每心跳一次,自管分片

```
execution_strategy: batch
run_pipeline(all_stations_context)
```

LISTEN/NOTIFY · 多实例无状态部署



没有引入 Redis / MQ

只用 PostgreSQL 自带能力实现跨实例点对点唤醒与广播订阅。运维更简单、成本更低、故障面更小。

亮点深化 · 启动时契约校验 · 可观测性原生集成

从「黑盒运行」变成「可运维对象」

Startup-time 契约校验

- sense / decide / act 是否 async
- handler 签名是否匹配 phase
- YAML 配置项是否完整合法
- 数据池 schema 是否可访问

Fail Fast

配置错误在 uvicorn 启动阶段就抛出,而不是跑到第一个 cycle 才炸。

这意味着 14 个 Agent × 多站的复杂配置,在上线前就能暴露所有契约违反,生产环境从不会出现「启动半小时后莫名其妙挂掉」的诡异故障。

Prometheus · 四大核心指标

`cycle_duration`

histogram by agent_id

每个 Agent 每轮耗时分布,慢 Agent 一眼定位

`cycle_total`

counter by (agent_id, status)

成功 / 失败计数,SLO 监控基础

`trigger_wake_total`

counter by (agent_id, reason)

心跳 / 消息 / 订阅触发占比,一眼看出事件驱动还是心跳驱动

`signal_pending`

gauge

下游消费落后的早期信号,背压指示器

Audit-by-Design

审计合规原生友好 · 为强监管行业而生的架构

在能源、金融、政务场景里,审计能力不是「锦上添花」,是**准入门槛**。

能源央企

- 国资委合规审计
- 碳资产核查
- 电力交易回溯

金融监管

- 银保监现场检查
- 反洗钱审计
- 信贷决策可解释

政务数据

- 可信可控可追溯
- 数据要素流通
- 算法可解释性要求

「没有就进不去」—— 对通用 Agent 框架而言,这是结构性缺失;对 SDAFI,这是架构副产物。

五阶段 → 审计证据链映射

每一次 S→D→A→F→I 循环,都原生产出完整、结构化、可回放的证据

S Sense	D Decide	A Act	F Feedback	I Improve
基于什么数据?	*怎么决策的?*	*影响了什么?*	*效果怎么样?*	*怎么修正?*
数据池快照	**Prompt + 规则**	**业务池 + 信号**	**Deviation**	**参数变更历史**
timestamp 精确到秒	决策输入全量记录	下游影响全追踪	真实值 vs 预测值偏差	calibration 调整轨迹
PG TABLE	PG TABLE	PG TABLE	PG TABLE	PG TABLE
device_pool · revenue_pool	cycle_log	agent_signal_log	feedback_log	improve_log

完整可回放的证据链:任意一次决策 → 可反向还原输入、过程、影响、修正。不是事后补的审计模块,是架构本身的副产物。

审计能力 · 同类框架对比

为什么只有 SDAFI 能原生支撑监管级回溯

审计维度	LangChain	LangGraph	SDAFI
决策输入留痕 *基于什么数据做的?*	✗ 内存瞬态	△ checkpoint 粗粒度	✓ 数据池快照带 timestamp
决策过程回放 *Prompt / 规则命中?*	✗ 需额外日志	△ 状态快照	✓ cycle_log 完整记录
协作因果链 *谁触发了谁?*	✗ 无结构化记录	△ 节点间 message	✓ agent_signal_log 全量
效果验证轨迹 *事后效果如何?*	✗ 不在设计范围	✗ 不在设计范围	✓ feedback deviation
自修正历史 *参数怎么变的?*	✗ 无此概念	✗ 无此概念	✓ improve_log
跨实例审计一致性 *多节点部署统一视图?*	✗ 难以保证	△ 需外部协调	✓ PG 单一真相源

✓ 原生支持　　△ 需自行实现　　✗ 不在设计范围

在其他场景里 SDAFI 未必最优,但在需要监管级回溯的业务里,它是目前唯一能开箱即用满足要求的 Agent 运行时。

三大强监管场景 · 适用性展开

从「技术选型优势」到「监管准入门槛」的定位跃迁

能源央企

合 规 要 求

- 发改委/国资委合规审计:调度决策全程留痕
- 碳资产核查:碳排放计量数据可追溯到原始来源
- 电力交易回溯:争议时段决策证据链举证
- 事故调查:任意时点系统状态可精确还原

适 用 价 值

进入省级电网 / 大型发电集团核心系统的必要条件

金融监管

合 规 要 求

- 银保监现场检查:信贷决策模型输入输出留痕
- 反洗钱审计:可疑交易判定链路可查
- 算法可解释性:《个人信息保护法》合规要求
- 穿透式监管:数据来源 + 模型版本 + 决策结果

适 用 价 值

满足《AI 金融应用指引》,普惠 / 小微贷决策落地前提

政务数据

合 规 要 求

- 「可信可控可追溯」:数据要素流通核心要求
- 公共数据授权运营:使用方可审计数据流向
- 算法备案:模型决策过程备查
- 跨部门协同:多主体责任边界可追踪

适 用 价 值

数据要素 × 普惠金融、政务智能体项目的合规底座

在这些场景下,SDAFI 的 Audit-by-Design 不是「更好的选项」,而是「别人做不到」。

与同类框架对比

SDAFI 不和 LangChain / LangGraph / Airflow 抢生态 —— 目标赛道完全不同

对比维度	SDAFI	LangGraph / CrewAI	Airflow / Prefect
Agent 定位	**业务领域对象**	LLM + 工具封装	任务节点
协作方式	**结构化信号**	自然语言协商	DAG 拓扑
触发来源	**心跳 / 消息 / 订阅**	用户请求驱动	调度器驱动
自主性	**自驱 · I 阶段自调整**	被动执行	被动执行
状态持久化	**原生数据池**	需外部集成	元数据表
审计证据链	**架构副产物**	需自建	仅任务日志
最适合场景	**严肃业务运营**	chatbot / copilot	ETL / 数据管道

SDAFI 的取舍:业务正确性 > 涌现能力。 不支持 Agent 间自然语言协商*(缺陷也是优点)*,但保证每次交互都是结构化、可审计、可回溯的。

首个闭环已验证 · solar_forecast 案例

能源场景率先跑通 S→D→A→F→I 全链路 · 同样的架构可平移到金融 / 政务场景

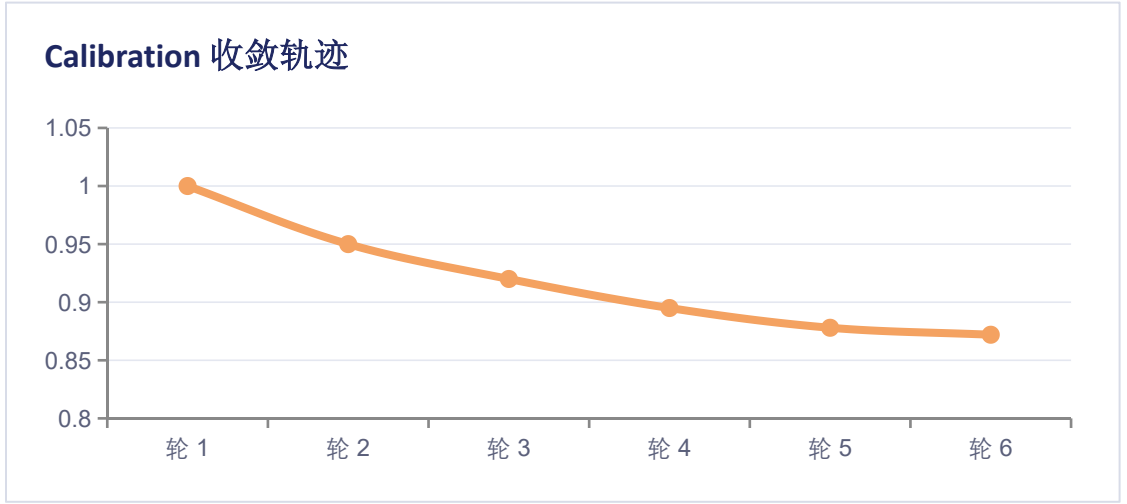
真实数据 · 非 mock

-27%

deviation

昨日预测
689 kWh

实际发电
506 kWh



验证流程(F → I → A 的完整链路)

F 算出 deviation

昨日预测 689 vs 实际 506 = -27%

I 调整 calibration

1.000 → 0.950

A 预测值乘 calibration

下一轮 A 写入自动修正

约 6 轮收敛

对着真 Postgres 跑出的数字

意义 · SDAFI 的 Agent 具备在线学习能力——同一套架构,能源首发,信贷 / 政务可直接复用。

落地数据 · 工程成熟度

ST-GZ-001 + ST-SZ-001 + ST-SZ-002 三站 dev 环境的真实数字

13	14	15+	151
Active Agents	**YAML 配置文件**	**Hypertables**	**单元 + 集成测试**
4 设备 · 5 绿电 · 2 财务 · 1 主控 · 1 区域	声明式,新增 Agent 无需改框架	TimescaleDB 支撑所有时序数据	工程成熟度到位,不是 demo

五阶段闭环实现进度

所有阶段 (S→D→A) *全部 Agent 跑通前三阶段*	13 / 13	100%
有真 feedback() 的 Agent *solar · load · price · dispatch*	4 / 13	30%
有真 improve() 的 Agent *solar_forecast 首个闭环*	1 / 13	8%

SUMMARY

SDAFI

= 严肃业务的 Agent 运行时

用最小的抽象

5 阶段 · 3 触发 · 1 介质 · 0 直接调用

支撑最大的业务复杂度

能源光储充 · 金融普惠信贷 · 政务数据要素

把「AI 原生运营」落到可运行的代码

不是 PPT 概念,是真 Postgres 跑出的数字

*在严肃业务里,**业务正确性 + 可审计性 > 涌现能力。** 这是 SDAFI 与通用 Agent 框架的根本差异。*

GET IN TOUCH

合作邀请

把你的业务场景,变成可运行、可审计的 Agent 系统

商务合作

POC 试点 · 行业解决方案 · 战略合作

技术交流

架构评审 · 深度技术分享 · 联合研究

官方渠道

官网 · 微信公众号「INSIDE AI」· 商务邮箱

*我们邀请你一起,**把 PPT 概念,变成严肃业务里真实跑起来的生产系统。** 期待在严肃业务里与你相遇。*